

תזמור סוכני בינה מלאכותית: מ-Prompting ל-Crews מוכנים לייצור

ארכיטקטורה, תזמור, והפקת מסמכים דטרמיניסטית

Amr Safadi

AI Agent Orchestration
מרצה: Dr. Yoram Segal

04-06-2026

תוכן העניינים

1	מ-Prompts למערכות סוכנים (From Prompts to Agent Systems)	3
1.1	2026 כנקודת מפנה: מ-helper לשכבה תפעולית	3
1.2	מדוע prompt בודד וארוך נכשל	3
1.3	תזת הספר: פירוק לצעדים תחומים, ניתנים לבדיקה ולשליטה	4
1.4	איורים, טבלה ונוסחה	4
1.5	מעבר דו-כיווני בין עברית לאנגלית	4
2	אבני הבניין של CrewAI	7
2.1	Agent: עובד דיגיטלי בעל תפקיד	7
2.2	Task: יחידת עבודה מדידה	7
2.3	Crew, Process ו-Context: הרכבה ודבק	8
3	תזמור סדרתי הלכה למעשה (Sequential Orchestration in Practice)	9
3.1	מדוע Sequential Process מתאים להפקת מסמכים	9
3.2	חמשת ה-Agents המתמחים ותפקידיהם הנפרדים	9
3.3	העברת artifacts מובנים כ-Context לאורך השרשרת	10
4	The Deterministic Harness — תשתית דטרמיניסטית סביב הסקת המודל	11
4.1	מהו Harness ולמה הוא דטרמיניסטי	11
4.2	ציטוטים, graph ו-validation כרכיבים דטרמיניסטיים	11
4.3	מדוע דטרמיניזם חיוני ל-reproducibility ולפלט אמין	12
5	הפקת מסמכים מקצועיים באמצעות LaTeX	13
5.1	מ-artifacts מאומתים ל-LaTeX: templates ו-escaping	13
5.2	כתיבה דו-כיוונית עברית-אנגלית עם מנוע Unicode	13
5.3	ביבליוגרפיה וקומפילציה רב-מעברית: מדוע citations דורשים כמה מעברים	14

15	6 מהוכחת היתכנות לסביבת Production
15	6.1 הפער השקט בין Demo ל-Production
15	6.2 אבטחת Agents ומודעות לעלות
16	6.3 מחסנית מודולרית ולאן הארכיטקטורה ממשיכה

פרק 1

מ-Prompts למערכות סוכנים (From) (Prompts to Agent Systems)

1.1 2026 נקודת מפנה: מ-helper לשכבה תפעולית

במשך שנים נתפסו מודלי שפה ככלי עזר (helper) שמסייע למשתמש האנושי לנסח טקסט, לסכם מסמך או להציע קטע קוד, כאשר האדם נשאר תמיד הגורם שמבצע את הפעולה בפועל. שנת 2026 מסמנת נקודת מפנה (inflection point) משום שה-Agent חדל להיות נספח לממשק והופך לשכבה תפעולית (operational layer) בתוך אפליקציות ארגוניות. במקום שאדם יקרא תשובה ויעתיק אותה ידנית, ה-Agent קורא נתונים ממערכות פנימיות, מפעיל tools, כותב לבסיסי נתונים ומפעיל תהליכים עסקיים מקצה לקצה. השינוי אינו רק טכנולוגי אלא ארכיטקטוני: הסוכן ממוקם בנתיב הקריטי של זרימת העבודה, ולכן נדרשת ממנו אותה רמת אמינות, observability ו-validation שמצופה מכל רכיב תוכנה תפעולי. ארגונים שמטמיעים אותם מצפים כיום לא רק לתשובה משכנעת אלא להתנהגות צפויה, ניתנת לניטור ולחזרה (reproducible). מכאן נובע הצורך במסגרות כמו CrewAI, המגדירות במפורש Agent, Task, Crew ו-Process כדי לתחום אחריות ולאפשר פיקוח. המעבר משכבת עזר לשכבה תפעולית מחייב מהנדסים לחשוב על סוכנים לא כעל קסם גנרטיבי אלא כעל רכיבי מערכת מתוזמרים, שיש להם קלט מוגדר, פלט מאומת והקשר (Context) מבוקר. זוהי המהפכה שהפרק הזה בא לתאר ולבסס.

1.2 מדוע prompt בודד וארוך נכשל

הפיתוי הראשוני בבניית מערכת מבוססת מודל שפה הוא לדחוס את כל הדרישות ל-prompt בודד וארוך: להסביר את המשימה, לצרף את כל הנתונים, לפרט את הכללים ולקוות שהמודל יפיק תוצאה שלמה במכה אחת. בפועל גישה זו קורסת ככל שהמשימה גדלה במורכבותה. ראשית, חסר planning: המודל מתבקש להחזיק במקביל את התכנון, הביצוע והבדיקה, ולכן הוא נוטה לדלג על שלבים או לסתור את עצמו. שנית, חסר

memory: בלא מנגנון זיכרון מפורש כל אינטראקציה מתחילה מאפס, ומידע שנצבר בצעד קודם נשמט. שלישית, חסרים tools: מודל מבודד אינו יכול לקרוא קובץ עדכני, להריץ חישוב מדויק או לפנות ל-API חיצוני, ולכן הוא ממציא עובדות במקום לאחזר אותן. ומעל הכל חסרה שכבת בקרה, ה-Harness, שתפקידה לתזמר את הצעדים, לאכוף סכמות, להפעיל validation על כל פלט ולהבטיח שגיאה בצעד אחד לא תזהם את כל התהליך. ה-Harness הוא מה שהופך אוסף קריאות למודל למערכת תפעולית אמינה: הוא קובע סדר, מטפל בכשלים, מספק observability ומאפשר חזרתיות. לכן prompt בודד אינו נכשל בגלל חולשת המודל, אלא בגלל היעדר ארכיטקטורה סביבו. המסקנה היא שמערכת סוכנים רצינית דורשת הפרדה מפורשת בין תכנון, זיכרון, כלים ובקרה.

1.3 תזת הספר: פירוק לצעדים תחומים, ניתנים לבדיקה ולשליטה

התזה המרכזית של ספר זה היא שכדי להפוך משימה גנרטיבית רחבה למערכת אמינה יש לפרק אותה לצעדים תחומים (bounded), ניתנים לבדיקה (inspectable) וניתנים לשליטה (controllable). במקום לבקש מן המודל להפיק תוצר ענק במכה אחת, אנו מגדירים שרשרת של Task יחידות, שלכל אחת קלט מוגדר, פלט בעל סכמה ברורה ואחריות מצומצמת. צעד תחום פירושו שהיקפו ידוע מראש ושאפשר להעריך את הצלחתו במנותק מן השאר. צעד ניתן לבדיקה פירושו שהפלט שלו אינו קופסה שחורה אלא ארטיפקט שאפשר לפתוח, לאמת מול validation ולהשוות לציפייה, כך ש-observability נבנית לתוך המערכת ולא מודבקת עליה בדיעבד. צעד ניתן לשליטה פירושו שה-Harness יכול לעצור, לחזור על צעד, להחליף Context או לחסום פלט שאינו עומד בכללים, לפני שהוא זורם הלאה. כך, למשל, הפקת מסמך LaTeX מורכב הופכת לרצף של צעדים שכל אחד מהם מיוצר ונבדק בנפרד, במקום ניסיון יחיד ושביר. גישה זו ממירה את אי הוודאות הגנרטיבית במבנה הנדסי: ה-Agent עדיין מספק את היצירתיות, אך ה-Process וה-Crew מספקים את המשמעת. שאר הספר מפתח את העיקרון הזה לכדי דפוסי תכן מעשיים, ומראה כיצד בונים מערכות סוכנים שניתן לסמוך עליהן בסביבה תפעולית אמיתית. לקריאה נוספת בנושאי פרק זה, ראו [1].

1.4 איורים, טבלה ונוסחה

פרק זה מסתמך על תיעוד ה-[1] framework.

$$Q = \sum_{i=1}^n w_i \cdot s_i \quad (1.1)$$

1.5 מעבר דו-כיווני בין עברית לאנגלית

הטקסט הראשי כתוב בעברית (RTL), ומונחים טכניים נשמרים באנגלית (LTR). לדוגמה, משפט שלם באנגלית: The system combines planning, research, writing, and

AI Agent Orchestration

איור 1.1: המחשה רעיונית של תזמור סוכנים (agent orchestration).

BookGen Sequential Agent and Harness Pipeline



איור 1.2: צינור סוכנים סדרתי עם Harness דטרמיניסטי.

review under a deterministic harness. כאן מודגם המעבר התקין בין כיוון הכתיבה מימין-לשמאל לכיוון משמאל-לימין.

תחום אחריות	Agent
תכנון מבנה המסמך	Planner
מחקר מותאם למקורות	Research
כתיבת כתב היד	Writer
ביקורת עריכה	Reviewer
מפרט הרכבה ל-LaTeX	LaTeX

טבלה 1.1: תחומי האחריות של כל Agent ורכיב דטרמיניסטי.

פרק 2

אבני הבניין של CrewAI

2.1 Agent: עובד דיגיטלי בעל תפקיד

ב-CrewAI ה-Agent אינו עוד קריאה גולמית למודל שפה, אלא ישות מובנית המגלמת עובד דיגיטלי בעל זהות ברורה. כל Agent מוגדר באמצעות שלושה רכיבים מרכזיים המעצבים את התנהגותו: role המתאר את התפקיד המקצועי שהוא ממלא, למשל חוקר, מהנדס תוכנה או עורך לשוני; goal המנסח את המטרה הקונקרטית שאליה הוא חותר; ו-backstory המספק רקע ונקודת מבט המעניקים לו אישיות, סגנון והקשר התנהגותי. הצירוף של שלושת אלה אינו קישוט ספרותי בלבד, אלא מנגנון מעשי המעצב את הפרומפט (prompt) הפנימי, ממקד את ההיגיון של המודל ומפחית סטייה מן המשימה. מעבר להגדרה הטקסטואלית, ל-Agent מוצמדים tools – כלים חיצוניים כגון חיפוש ברשת, קריאת קבצים, הרצת קוד או פנייה ל-API – המרחיבים את יכולותיו אל מעבר לטקסט שהמודל יודע לייצר מתוך עצמו. כך ה-Agent הופך לישות הפועלת בעולם ולא רק מדברת עליו. ההבחנה בין Agent ל-model call פשוטה אך עקרונית: model call הוא אירוע חד-פעמי וחסר זיכרון, ואילו Agent הוא יחידה בעלת תפקיד מתמשך, יכולת מוגדרת ואחריות ברורה בתוך הצוות. תפיסה זו מאפשרת לחלק מערכת מורכבת לעובדים מתמחים, שכל אחד מהם אחראי לתחום צר ומוגדר, ובכך משפרת מודולריות, שקיפות ויכולת תחזוקה של הפתרון כולו.

2.2 Task: יחידת עבודה מדידה

ה-Task הוא היחידה האטומית של העבודה ב-CrewAI, והוא מתרגם כוונה כללית למשימה קונקרטית, מוגדרת ומדידה. כל Task נשען על שני רכיבים שאי אפשר לוותר עליהם: description, המתאר במדויק מה צריך להיעשות ואילו שיקולים יש לקחת בחשבון, ו-expected output, המגדיר במפורש כיצד נראית תוצאה ראויה ומה צורתה הצפויה. ההגדרה המפורשת של expected output היא לב העניין, משום שהיא הופכת את המשימה ממשהו עמום למשהו שניתן לאמת מולו (validation). כאשר התוצר מצופה להיות, למשל,

טבלה בפורמט מסוים, קובץ LaTeX תקין או רשימת ממצאים מובנית, הצוות יכול לבדוק האם הפלט עומד בציפייה במקום להסתפק בהתרשמות סובייקטיבית. כל Task משויך בדרך כלל ל-Agent מסוים האחראי לביצועו, ובכך נוצר חיבור ישיר בין התפקיד לבין העבודה שהוא מבצע בפועל. ה-Task יכול גם לקבל הקשר מ-tasks שקדמו לו ולהזין את אלה שיבואו אחריו, ובכך הוא הופך לחוליה בשרשרת ולא לאי מבודד. הגדרה זו של עבודה כיחידות מדידות תומכת ישירות ב-observability: כל שלב ניתן למעקב, לרישום ולבחינה, וכשל מקומי ניתן לאיתור ולתיקון בלי לפרק את המערכת כולה. כך CrewAI מאפשר לבנות תהליכים אמניים שבהם ההצלחה אינה עניין של מזל אלא תוצאה של דרישות ברורות וקריטריונים שניתן למדוד מולם בכל ריצה.

2.3 Crew, Process ו-Context: הרכבה ודבק

לאחר שהוגדרו Agents ו-Tasks, נדרש מנגנון שירכיב אותם לכדי מערכת פועלת אחת, וכאן נכנסים לתמונה Crew, Process ו-Context. ה-Crew הוא המעטפת המאגדת קבוצה של agents ואת ה-tasks המוטלים עליהם ליחידת עבודה אחת בעלת מטרה משותפת, בדומה לצוות אנושי שחבריו תורמים כל אחד את התמחותו. ה-Process מגדיר את אופן ההפעלה של אותו צוות: בתהליך sequential ה-tasks מתבצעים בזה אחר זה לפי סדר קבוע, ואילו בתהליך hierarchical מתמנה מנהל המחלק ומתאם את העבודה בין החברים. בחירת ה-Process קובעת את אופי התיאום ואת זרימת השליטה בין השלבים. הרכיב המאחד את הכול הוא ה-Context, המשמש כדבק בין השלבים. ה-Context הוא המידע העובר משלב לשלב: התוצר של Task אחד הופך לקלט או לרקע של ה-Task הבא, כך שכל שלב נבנה על גבי מה שקדם לו במקום להתחיל מאפס. בלי Context הצוות היה אוסף של עובדים מנותקים שאינם מודעים זה לעבודתו של זה; עם Context נוצרת שרשרת קוהרנטית שבה הידע מצטבר וזורם. כך ההרכבה של agents ו-tasks בתוך Crew, יחד עם Process מתאים ו-Context רציף, היא שמאפשרת ל-CrewAI להפוך רכיבים בודדים לתזמור (orchestration) של ממש, שבו השלם גדול מסכום חלקיו ומסוגל לבצע משימות מורכבות מקצה לקצה.

לקריאה נוספת בנושאי פרק זה, ראו [2].

פרק 3

תזמור סדרתי הלכה למעשה Sequential Orchestration in) (Practice

3.1 מדוע Sequential Process מתאים להפקת מסמכים

הפקת מסמך עיון מקצועי אינה פעולה אחת אלא רצף של שלבים שבו כל שלב נשען על תוצר השלב שקדם לו, ולכן Sequential Process ב-CrewAI הוא הבחירה הטבעית עבורו. אי אפשר לחקור נושא לפני שקיים book plan שממקד את החקירה, אי אפשר לכתוב manuscript לפני שה-research pack מספק את החומר, אי אפשר לבצע review לפני שקיים draft, ואי אפשר לבנות latex spec לפני שהתוכן עבר ביקורת והתייצב. תלות חד-כיוונית זו היא בדיוק מה ש-Sequential Process מבטא: ה-Crew מריץ Task אחר Task בסדר קבוע, וה-Context של כל Task מורכב מתוצרי ה-Tasks שלפניו. היתרון המעשי הוא שהזרימה נעשית גלויה וניתנת לבדיקה: בכל נקודה ברור איזה artifact כבר קיים ומהי הכניסה הצפויה לשלב הבא. בניגוד ל-hierarchical process, שבו Manager Agent מנתב עבודה באופן דינמי, ה-Sequential Process פשוט יותר להסבר, צפוי יותר בהתנהגותו, וקל יותר ל-observability ול-validation. בפרויקט זה הוא גם משקף עיקרון הנדסי חשוב: עבודה דטרמיניסטית כמו citation management, יצירת assets, רינדור LaTeX והידור PDF נשארת מחוץ ל-Crew ורצה ב-Harness לאחר חמשת ה-Agent Tasks. כך מתקבל גבול ברור בין שיקול דעת של מודל לבין צעדים ניתנים לאימות, וכל סטייה מהסדר הצפוי נחשפת מיד.

3.2 חמשת ה-Agents המתמחים ותפקידיהם הנפרדים

המערכת מורכבת מחמישה Agents שכל אחד מהם מוגדר באמצעות role, goal ו-backstory, ובאופן מכוון תחומי האחריות שלהם אינם חופפים. ה-Planner Agent מתפקד כעורך טכני

בכיר: הוא ממיר את הנושא ואת דרישות המטלה ל-book plan הכולל מבנה פרקים, מיקום של image, graph, table ו-formula, ו-acceptance checklist, אך אינו כותב פרוזה. ה-Research Agent הוא אנליסט מחקר שאוסף מושגי מפתח, הגדרות ומועמדים למקורות, ורק לו מותר להשתמש בכלי חיפוש חיצוניים; הוא מציע source candidates אך אינו מייצר את ה-bibliography הסופי. ה-Writer Agent הוא כותב טכני בכיר שעובד מתוך ה-Context בלבד, ללא חיפוש עצמאי, וממיר את התוכנית והמחקר ל-manuscript עם citation markers ו-placeholders. ה-Reviewer Agent הוא עורך ובוחר דרישות: הוא משפר בהירות ודיוק מבלי לשנות את המשמעות, ומפיק review report עם requirement checklist ואזהרות. ה-LaTeX Agent מתכנן את ה-assembly ומפיק latex spec, אך אינו מריץ קומפילר ואינו אחראי ל-build correctness. ההפרדה החדה הזו, המעוגנת בסעיפי Not Responsible For Agent, מונעת התנגשות תפקידים, שומרת על מערכת קטנה הניתנת להסבר, ומבטיחה שכל החלטה תיפול במקום אחד ויחיד בשרשרת.

3.3 העברת artifacts מובנים כ-Context לאורך השרשרת

הדבק שמחבר את חמשת ה-Tasks הוא ה-Context: כל Agent אחרי ה-Planner צורך במפורש את תוצרי השלבים שקדמו לו, וההעברה נעשית באמצעות artifacts מובנים ולא כטקסט חופשי. ה-Planner מפיק את book plan, שמשמש Context ל-Research Task; ה-Research Agent מפיק את research pack, ויחד עם ה-book plan הוא מהווה את ה-Context של ה-Writing Task; ה-Writer מפיק את ה-manuscript, וה-Reviewer מקבל את שלושת התוצרים יחד ומחזיר reviewed manuscript ו-review report; לבסוף ה-LaTeX Agent צורך את ה-book plan, את ה-reviewed manuscript ואת ה-review report כדי לבנות את ה-latex spec. שרשרת מצטברת זו מבטיחה שהחלטות מוקדמות, כמו ה-acceptance checklist של ה-Planner, נשארות זמינות ונבדקות עד סוף הזרימה. הבחירה ב-artifacts מובנים, רובם בפורמט JSON עם expected output מוגדר לכל Task, היא מה שמאפשר validation דטרמיניסטי לאחר שה-Crew מסיים: ה-Harness יכול לוודא שכל citation marker מתאים למקור ידוע, שה-latex spec מפנה לקבצים תקפים, ושכל דרישת המטלה מכוסה. כך ה-Context אינו רק מנגנון תקשורת בין Agents אלא גם שכבת חוזה הניתנת לאכיפה, ההופכת את ה-Sequential Orchestration למערכת שקופה, ניתנת למעקב וניתנת לאימות מקצה לקצה. לקריאה נוספת בנושאי פרק זה, ראו [3].

פרק 4

The Deterministic Harness – תשתית דטרמיניסטית סביב הסקת המודל

4.1 מהו Harness ולמה הוא דטרמיניסטי

בלב הארכיטקטורה שלנו ניצב רעיון פשוט אך מכריע: סביב הסקת המודל יש לעטוף שכבת תוכנה דטרמיניסטית הקרויה Harness. מודל שפה הוא רכיב הסתברותי מטבעו – בהינתן אותו prompt הוא עשוי להחזיר פלטים שונים, להמציא תחביר, או לחרוג מהמבנה הצפוי. לכן אנו מפרידים בחדות בין שתי שכבות אחריות. ה-Agents של CrewAI – Planner, Research, Writer, Reviewer ו-LaTeX – מבצעים עבודה אינטלקטואלית הדורשת judgment: תכנון, סינתזה של מחקר, כתיבת פרוזה ועריכה. לעומתם, ה-Harness מבצע את העבודה השבירה והניתנת לאימות באמצעות קוד Python קבוע: reconciliation של ציטוטים, יצירת graph, validation, escaping של תווים ו-compilation של ה-PDF. ההשראה לכך מגיעה מתפיסת ה-production harness שבה Harness אמיתי עוטף את קריאות המודל ב-tools, ב-context, ב-parsing, ב-validation, ב-observability וב-guardrails. ההיגיון הוא שכל פעולה שתוצאתה חייבת להיות נכונה ובת-שחזור אסור שתסמוך על אלתור של מודל. כך, פלט ה-Agent נשמר כ-artifact מובנה ובר-בדיקה – לדוגמה book_plan או manuscript – וה-Harness צורך אותו, מאמת אותו, וממיר אותו לתוצר סופי. תכן זה ממקם את אי-הוודאות במקום אחד מבוקר: בתוך הסקת המודל. כל מה שמסביב הופך לצינור הנדסי צפוי, שניתן להריץ שוב ושוב ולקבל בדיוק אותה התנהגות, ובכך להפוך מערכת יצירתית במהותה למערכת אמינה תפעולית.

4.2 ציטוטים, graph ו-validation כרכיבים דטרמיניסטיים

שלוש משימות בולטות מודגמות במערכת כרכיבים דטרמיניסטיים מובהקים – ולא כ-Agents. הראשונה היא ניהול הציטוטים. מודל אסור שימציא רשומות bibliography או citation keys, ולכן ה-Citation Manager הוא קוד Python הקורא source registry אוצר

ומבוקר, מייצר קובץ references.bib תחבירי תקין עם escaping בטוח, ומבצע reconciliation המוודא שכל citation marker ב-manuscript נפתר למקור מוכר. אם מופיע מפתח שאינו מזוהה, התהליך נכשל בקול רם עוד לפני ה-compilation. עיקרון מפתח כאן הוא idempotency: אותו registry מפיק תמיד בדיוק אותו .bib. המשימה השנייה היא יצירת ה-graph. השרטוט המתאר את ה-Planner — pipeline אל Research אל Writer אל Reviewer אל LaTeX ואל רכיבי הבנייה הדטרמיניסטיים — נוצר בקוד Python קבוע ולא על ידי Agent, שכן graph דטרמיניסטי הוא בר-שחזור וקל ל-validation. המשימה השלישית היא ה-validation עצמה. ה-Validator הדטרמיניסטי בודק שכל הרכיבים הנדרשים קיימים לפני ה-compilation: cover metadata, TOC, chapters, sections, לפחות תמונה אחת, קובץ ה-graph, table, נוסחת display שאינה טקסט רגיל, מקטע BiDi עברית-אנגלית, citation keys תקפים וקובצי ה-LaTeX source. החלוקה הזו עקבית עם תפיסת הקורס: עבודה השופטת איכות ומשמעות נשארת אצל Agent כמו ה-Reviewer, אך הבדיקה הטכנית — האם הכול מחובר, מתקמפל ותקין — חייבת להיות דטרמיניסטית, שכן זו בדיקה הנדסית ולא תוכנית בלבד, והיא ניתנת ל-testing אובייקטיבי.

4.3 מדוע דטרמיניזם חיוני ל-reproducibility ולפלט אמין

הדטרמיניזם איננו קישוט הנדסי אלא תנאי יסוד לשלושה ערכים שבלעדיהם המערכת אינה ראויה לאמון. הראשון הוא reproducibility. כאשר רכיבי ה-Harness מתנהגים באופן צפוי, הרצה חוזרת על אותם artifacts מפיקה בדיוק את אותו פלט — אותו references.bib, אותו graph ואותו validation report. תכונה זו חיונית במיוחד כשמדובר בתוצר אקדמי מדורג, שבו בודק חייב להיות מסוגל לשחזר את הבנייה ולקבל את אותו final.pdf. הערך השני הוא testing. רכיב הסתברותי קשה לבדיקה, שכן אין לו פלט יחיד נכון; אך רכיב דטרמיניסטי ניתן לכסות ב-unit tests מדויקים: לוודא ש-citation key לא מזוהה מדווח כ-unresolved, שתווים מיוחדים עוברים escaping, ושאותו registry מפיק שוב את אותו .bib. כך ה-schema validation, citation validation, asset — quality gates validation ו-LaTeX validation — הופכים לבדיקות אוטומטיות אמינות במקום הסתמכות על שיפוט אנושי. הערך השלישי הוא trustworthy output. על ידי מיקום אי-הוודאות אך ורק בתוך הסקת המודל, ועטיפתה ב-validation וב-guardrails, אנו מבטיחים שתקלה של Agent — מפתח שגוי, נכס חסר או נוסחה פגומה — נתפסת מוקדם ובאופן גלוי, ולא מתפשטת בשקט אל המסמך הסופי. שילוב יצירתיות הסתברותית של מודל עם תשתית דטרמיניסטית קפדנית הוא שמאפשר למערכת להיות בו-זמנית גמישה בתוכן ואמינה בתוצר, וזהו בדיוק האיזון שתזמור סוכנים מקצועי שואף אליו. לקריאה נוספת בנושאי פרק זה, ראו [1].

פרק 5

הפקת מסמכים מקצועיים באמצעות LaTeX

5.1 מ-artifacts מאומתים ל-LaTeX: templates ו-escaping

לאחר שכל ה-Agents סיימו את עבודתם ושלבו ה-validation אישר את התוצרים, עומד לרשותנו אוסף של artifacts מובנים: כתב היד שעבר review, מטא-נתונים, נכסים גרפיים וקובץ מקורות. השלב הבא אינו יצירתי אלא דטרמיניסטי לחלוטין, ובכך טמון עיקרון מרכזי בתכנון המערכת. ה-Agent מתאר כוונה בלבד, באמצעות מסמך latex_spec מובנה הקובע איזה template לבחור, כיצד למפות פרקים וכיצד לשבץ נכסים, אך לעולם אינו כותב LaTeX גולמי בעצמו. את הרנדור מבצע רכיב Python ייעודי, ה-renderer, הממלא templates של Jinja2 בתוכן המאומת ומייצר את הקבצים main.tex ואת קבצי הפרקים. ההפרדה הזו בין שיקול דעת של ה-Agent לבין הפקה דטרמיניסטית היא לב ההבטחה לשחזוריות (reproducibility). אתגר קריטי הוא escaping: טקסט שמקורו ב-Agent הוא בגדר untrusted text, ותווים מיוחדים של LaTeX, כדוגמת סימני אחוז, אמפרסנד או סוגריים מסולסלים, עלולים לשבור את הקומפילציה או אף לאפשר הזרקת פקודות. לכן הרנדרר מחזיק במודול escaping ייעודי, המעבד כל מחרוזת לפני שיבוצה ב-template ומבטיח שתוכן שרירותי לא יהפוך לקוד פעיל. כך מתקבל קוד מקור תקני, אחיד וצפוי, שאינו תלוי בגחמות הניסוח של המודל. גישה זו גם תומכת ב-observability: כיוון שהרנדור ידוע ומבוקר, ניתן להריץ אותו גם ללא toolchain מותקן ולקבל קבצי tex תקפים לבדיקה, מה שמפריד בבירור בין שלב ההרכבה לשלב הקומפילציה בפועל.

5.2 כתיבה דו-כיוונית עברית-אנגלית עם מנוע Unicode

מסמך מקצועי בעברית המשלב מונחים טכניים באנגלית מציב אתגר טיפוגרפי של ממש, שכן עברית נכתבת מימין לשמאל ואילו מונחים כדוגמת Agent, Crew או validation נכתבים משמאל לימין באותה שורה. ניהול נכון של מצב זה, המכונה BiDi או bidirectional

typesetting, דורש מנוע מודרני המבוסס Unicode. מנוע pdfLaTeX הוותיק אינו מספק תמיכה אמينة בכך, ולכן המערכת בוחרת ב-LuaLaTeX כברירת מחדל, עם XeLaTeX-fallback. שני מנועים אלו קוראים ישירות קבצים בקידוד Unicode ויודעים לטעון גופנים מודרניים התומכים בעברית, מה שמייתר את שכבות הקידוד המסורבלות של העבר. את ניהול השפות מבצעת חבילת polyglossia יחד עם רכיב bidi, המחליפים את החבילות הישנות שלא הותאמו לעברית. polyglossia מאפשרת להגדיר שפה ראשית ושפה משנית ולעבור ביניהן באופן מפורש, כך שכל קטע טקסט מקבל את כיוון הכתיבה הנכון לו. העיקרון המעשי הוא בידוד: יש לעטוף כל ריצה של אנגלית בתוך משפט עברי כך שתישמר כיחידה רציפה משמאל לימין, ולמנוע מהאלגוריתם הדו-כיווני לערבב את סדר האותיות סביב נקודות המעבר. בקרת איכות מחייבת אימות חזותי של המעברים: יש לוודא שמספרים, סימני פיסוק וסוגריים נצמדים לצד הנכון, ושמונחים טכניים אינם נשברים. הרנדרר מקבל מטא-נתוני BiDi מתוך ה-latex_spec ומשבץ את מתגי השפה במקומות הנכונים, כך שגם תוצר רב-לשוני זה נשאר דטרמיניסטי וניתן לשחזור מלא.

5.3 ביבליוגרפיה וקומפילציה רב-מעברית: מדוע citations דורשים כמה מעברים

אחד ההיבטים הפחות אינטואיטיביים בהפקת מסמך LaTeX הוא שאי אפשר להפיק ביבליוגרפיה תקינה במעבר קומפילציה יחיד. הסיבה נעוצה באופן שבו LaTeX פותר הפניות: במעבר הראשון המהדר אינו יודע עדיין לאילו מקורות הטקסט מפנה, ולכן הוא רק אוסף את סימני ה-citation ורושם אותם בקובצי עזר, כדוגמת קובץ aux וקובץ bcf. רק לאחר מכן נכנס לפעולה ה-backend הביבליוגרפי, biber, הקורא את רשימת ההפניות שנאספה, מצליב אותה מול קובץ references.bib, וממין ומעצב את הרשומות לפי סגנון הציטוט הנדרש. המעברים הבאים של המהדר משלבים את הפלט הזה חזרה אל המסמך, וממלאים את מספרי ההפניות ואת רשימת המקורות במקומותיהם. לכן הרצף הקונוי במערכת הוא ריצת המהדר, ואחריה biber, ואחריה שתי ריצות נוספות של המהדר. ללא רצף זה יישארו במסמך סימני שאלה כפולים במקום מספרי citation, וגם cross-references פנימיים, כדוגמת הפניות לאיורים ולטבלאות, לא יפתרו. כאן בולט שוב עקרון ההפרדה: ניהול ה-citations עצמו דטרמיניסטי ואינו Agent, שכן אסור שמודל ימציא מפתחות או רשומות. קובץ ה-bib נוצר מתוך רישום מקורות מבוקר, ושלב reconciliation מאמת שכל מפתח ציטוט במסמך מתאים למקור מוכר, ונכשל בקול רם אם נמצא מפתח שאינו קיים. כך משלבת המערכת קומפילציה רב-מעברית עם validation מחמיר, ומבטיחה מסמך אקדמי אמין שבו כל הפניה נפתרת כראוי. לקריאה נוספת בנושאי פרק זה, ראו [2].

פרק 6

מהוכחת היתכנות לסביבת Production

6.1 הפער השקט בין Demo ל-Production

ההבדל בין Demo מרשים למערכת Production אינו טמון באיכות הפלט הבודד אלא בהתנהגות תחת שינוי, כשל ועומס. ב-Demo כל Agent מקבל קלט נקי, ה-Crew מריץ Process יחיד בנתיב מאושר, וה-Context מועבר באופן שצפוי מראש. אך ברגע שמערכת התזמור פוגשת קלטים אמיתיים, היא נחשפת לפער השקט שאיש אינו רואה במצגת: מה קורה כש-Task נכשל באמצע שרשרת, כשמודל מחזיר LaTeX שבור או JSON שלא עובר validation, או כששירות חיצוני קורס. מערכת Production חייבת recovery מובנה – ניסיון חוזר עם backoff, נתיב fallback, ומנגנון שמבודד כשל של Agent בודד כדי שלא יפיל את ה-Crew כולו. לצד ההתאוששות נדרשת monitoring רציפה: לא רק לוג בסוף הריצה אלא observability לכל שלב, עם מדדי זמן, שיעורי כשל, ועקיבות מלאה של ה-Context שעבר בין Tasks. שכבת ה-Harness הדטרמיניסטית הופכת כאן לקריטית, משום שהיא מאפשרת לשחזר ריצה, להשוות פלטים, ולזהות רגרסיה כשמודל או prompt משתנים. בלי control מפורש על השינוי – גרסאות נעולות של מודלים, בדיקות שמריצות תרחישים מוכרים לפני פריסה, ושערי validation שחוסמים פלט לא תקין – המערכת תתנהג היטב היום ותיכשל באופן בלתי צפוי מחר. המעבר ל-Production הוא במידה רבה המעבר מאופטימיזציה למקרה הטוב לעיצוב למקרה הגרוע, שבו כל רכיב מתוכנן להיכשל בחן ולהתאושש באופן נשלט.

6.2 אבטחת Agents ומודעות לעלות

כשמערכת תזמור עוברת ל-Production, כל Agent הופך למשטח תקיפה פוטנציאלי ולמקור הוצאה כספית, ושני הממדים הללו דורשים משמעת הנדסית. עקרון ה-least privilege הוא הבסיס: כל Agent מקבל אך ורק את הכלים, ההרשאות והגישה לנתונים שדרושים למשימתו, ולא יותר. Agent שתפקידו לעצב LaTeX אינו זקוק לגישה לרשת או למפתחות API חיצוניים, ו-Agent שמסכם נתונים אינו צריך הרשאת כתיבה למאגר. הפרדה

זו מצמצמת את הנזק כש-prompt injection או קלט עוין מצליחים לחדור. מעליה נדרשים guardrails – שכבות validation שבודקות גם את הקלט הנכנס וגם את הפלט היוצא, מסננות הוראות זדוניות שמוטמעות ב-Context, ומוודאות שהפעולות שה-Agent מבקש לבצע נמצאות בתחום המותר. במקביל, מודעות לעלות אינה מותרות אלא חלק מהארכיטקטורה: כל קריאה למודל צורכת tokens שעולים כסף, ושרשרת Tasks ארוכה עם Context נפוח עלולה לייקר ריצה בודדת פי כמה. משמעת של token-cost מחייבת מדידה לכל Task, תקצוב מקסימלי לריצה, בחירה מושכלת בין מודל יקר וחזק למודל זול וקל למשימות פשוטות, וצמצום ה-Context המועבר רק לרלוונטי. ניטור עלות לצד observability מאפשר לזהות Agent שנכנס ללולאה יקרה או prompt שמנפח צריכה. אבטחה ועלות הם שני צדדים של אותה משמעת: שליטה הדוקה במה שכל Agent רשאי לעשות ובמה שכל Agent מבזבז.

6.3 מחסנית מודולרית ולאן הארכיטקטורה ממשיכה

מערכת Production בוגרת נשענת על מחסנית מודולרית ומנוטרת, שבה כל רכיב – הגדרת Agent, Task, בחירת Process, שכבת ה-Harness ושערי ה-validation – ניתן להחלפה, לבדיקה ולניטור בנפרד. מודולריות זו אינה רק נוחות הנדסית אלא תנאי להתפתחות: כשהגבולות בין הרכיבים ברורים ו-observability מכסה כל שלב, ניתן לשדרג חלק אחד מבלי לסכן את השלם, ולהריץ ניסויים מבוקרים. מנקודה זו הארכיטקטורה ממשיכה בכמה כיוונים טבעיים. הראשון הוא hierarchical crews – במקום Crew שטוח שבו כל Agent שווה, נוצר Agent מנהל שמתזמר Agents כפופים, מפרק בעיה מורכבת לתת-משימות, ומקצה אותן דינמית. מבנה היררכי מאפשר התמודדות עם משימות פתוחות שאינן ניתנות לתכנון מראש ב-Process קווי. הכיוון השני הוא שילוב RAG, שבו ה-Agents שואבים ידע עדכני ומבוסס ממאגר חיצוני במקום להישען רק על משקלי המודל, מה שמספר דיוק ומצמצם hallucination בתחומי ידע ספציפיים. הכיוון השלישי, ואולי החשוב מכולם לאורך זמן, הוא evaluation שיטתי: מסגרת מדידה שמריצה את ה-Crew על אוסף תרחישים מייצגים, מציינת את איכות הפלטים מול קריטריונים מוגדרים, ומזהה רגרסיה לפני פריסה. בלי evaluation אובייקטיבי כל שיפור הוא ניחוש. שילוב של מחסנית מודולרית, RAG, hierarchical crews ו-evaluation הוא הנתבי שבו מערכת תזמור הופכת מכלי חד-פעמי לפלטפורמה שמתפתחת, נמדדת ומשתפרת באופן שיטתי. לקריאה נוספת בנושאי פרק זה, ראו [3].

ביבליוגרפיה

- CrewAI. CrewAI Documentation. Framework documentation for CrewAI agents, [1]
tasks, crews, and processes. 2026. URL
<https://docs.crewai.com/>
- LangChain. LangChain Documentation. Reference material for harness-style LLM [2]
application design. 2026. URL
<https://python.langchain.com/>
- The LaTeX Project. LaTeX Project Documentation. Reference documentation for [3]
professional LaTeX document preparation. 2026. URL
<https://www.latex-project.org/help/documentation/>